

CSCE 629 ANALYSIS OF ALGORITHMS · MODULE 08 · LECTURE NOTES

Minimum Spanning Trees and Union-Find

Two greedy algorithms that are both correct, and the structure that makes one of them fast.

MODULE OUTCOMES

- State and apply the cut property.
- Implement Kruskal and Prim and explain why both are correct.
- Implement union-find with path compression and union by rank.
- Explain the inverse Ackermann bound informally.
- Recognise the matroid structure that makes greedy work here.

1 The problem

Given a connected, undirected, weighted graph, find a spanning tree — a subset of edges connecting all vertices with no cycles — of minimum total weight. A spanning tree always has exactly $V-1$ edges.

There may be exponentially many spanning trees, so enumeration is hopeless. Two different greedy algorithms both find the optimum, which is unusual enough to deserve an explanation.

2 The cut property

STATEMENT

Let $(S, V-S)$ be any partition of the vertices into two non-empty sets — a **cut**. Let e be a minimum-weight edge with one endpoint in each part.

Then e belongs to some minimum spanning tree. If all edge weights are distinct, e belongs to *every* minimum spanning tree.

Proof. Let T be an MST not containing e . Adding e to T creates exactly one cycle. That cycle starts in S and ends in S while passing through $V-S$, so it crosses the cut an even number of times — in particular, at least once besides e . Call such another crossing edge f .

Since e is a minimum-weight edge across the cut, $w(e) \leq w(f)$. Form $T' = T - \{f\} + \{e\}$. Removing f breaks the cycle, so T' is acyclic; it still has $V-1$ edges and still connects everything, so it is a spanning tree. Its

weight is $w(T) - w(f) + w(e) \leq w(T)$. Since T was minimum, T' is minimum too, and it contains e . ■

This exchange argument — take a hypothetical optimum, swap one element for your greedy choice, show it does not get worse — is the standard proof technique for greedy correctness, and Module 09 generalises it.

3 Two algorithms, one lemma

3.1 Kruskal

```
def kruskal(V, edges):
    edges.sort(key=lambda e: e[2])          # by weight
    dsu = DSU(len(V))
    mst = []
    for (u, v, w) in edges:
        if dsu.union(u, v):                 # different components?
            mst.append((u, v, w))
    return mst
```

Sort all edges and add each one that connects two currently separate components. The structure grows as a forest that gradually merges into a single tree. $O(E \log E)$ for the sort, plus near-linear work for the union-find operations — so the sort dominates.

Why the cut property applies: when Kruskal adds edge e , consider the cut separating the component containing one endpoint of e from everything else. Every edge cheaper than e has already been examined, and any that crossed this cut would have merged the two sides. So e is a minimum-weight edge across that cut.

3.2 Prim

Start from an arbitrary vertex and repeatedly add the cheapest edge connecting the tree built so far to a vertex outside it. The structure is connected at every step.

The implementation is Dijkstra's with one change: the priority queue key is the weight of the connecting edge rather than the accumulated path distance. Noticing this saves learning a second algorithm.

Why the cut property applies: the cut is (vertices in the tree) versus (vertices outside), and Prim picks precisely the minimum-weight edge crossing it.

DIFFERENT CUTS, THE SAME LEMMA

Prim always applies the cut property to the cut between its growing tree and the rest of the graph. Kruskal applies it to whichever cut separates the component it is currently merging.

Two quite different-looking strategies turn out to be the same argument deployed differently, which is why both reach the optimum.

4 Union-find

Kruskal needs to answer 'are u and v already connected?' and 'merge these two components' many times. A **disjoint set union** structure does both in effectively constant amortised time.

```
class DSU:
    def __init__(self, n):
        self.p = list(range(n))      # parent pointers
        self.rank = [0] * n         # upper bound on tree height

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])  # path compression
        return self.p[x]

    def union(self, a, b):
        a, b = self.find(a), self.find(b)
        if a == b:
            return False
        if self.rank[a] < self.rank[b]:      # union by rank
            a, b = b, a
        self.p[b] = a
        if self.rank[a] == self.rank[b]:
            self.rank[a] += 1
        return True
```

4.1 The two optimisations

- **Union by rank.** When merging, hang the shallower tree under the deeper one. This keeps heights at $O(\log n)$ rather than letting a bad sequence of unions build a path.
- **Path compression.** During find, repoint every node on the path directly at the root. Each traversal flattens the structure it walked, so future queries are cheaper.

Optimisations	Per operation
Neither	$O(\log n)$, and $O(n)$ in the worst case without rank
Union by rank only	$O(\log n)$
Path compression only	$O(\log n)$ amortised
Both	$O(\alpha(n))$ amortised

THE INVERSE ACKERMANN FUNCTION

$\alpha(n)$ is the inverse of a function that grows so fast it is hard to describe. In the other direction, α grows so slowly that $\alpha(n) \leq 4$ for every n less than 2^{65536} — vastly more than the number of atoms in the observable universe.

So union-find is constant time for every input that will ever exist, while being provably not constant time. That distinction is a small delight and a good reminder that asymptotic statements are about limits, not about your data.

Tarjan also proved the bound is *tight*: no union-find implementation can achieve $O(1)$ worst-case amortised. This is one of the few data structures with a matching lower bound.

5 Why greedy works here

Greedy algorithms usually fail. They are wrong for shortest paths with negative edges, wrong for 0/1 knapsack, wrong for set cover. Their success on MST is not luck, and there is a precise characterisation of when greed is safe.

A **matroid** is a ground set E with a family of 'independent' subsets satisfying two conditions: every subset of an independent set is independent, and if A and B are independent with $|A| < |B|$, then some element of B can be added to A keeping it independent (the *exchange property*).

The acyclic edge subsets of a graph — its forests — form a matroid, the graphic matroid.

RADO-EDMONDS

The greedy algorithm finds an optimal solution for **every** weight function precisely when the underlying structure is a matroid.

This is a genuine characterisation, not merely a sufficient condition. So 'is this a matroid?' is the right question to ask when wondering whether a greedy approach can be made correct — and when the answer is no, as for knapsack, no amount of cleverness in the greedy rule will fix it.

That is the moment to reach for dynamic programming (Module 10) or approximation (Module 13).

6 In practice

Situation	Choice	Reason
Sparse graph ($E = O(V)$)	Kruskal	The sort dominates and E is small.
Dense graph ($E \approx V^2$)	Prim with an array	$O(V^2)$ without heap overhead beats $O(E \log E)$.
Edges already sorted, or integer weights	Kruskal	Skip or linearise the sort; the rest is near-linear.

Situation	Choice	Reason
Need a connected structure at every step	Prim	Kruskal's intermediate state is a forest, not a tree.

6.1 What MSTs are actually used for

- **Single-linkage clustering.** Build the MST and delete the $k-1$ heaviest edges: the remaining components are exactly the k clusters. This equivalence is why MST algorithms appear in clustering libraries.
- **Image segmentation.** Pixels as vertices, colour difference as edge weight; the MST structure reveals region boundaries.
- **Metric TSP approximation.** A depth-first walk of the MST visits every vertex and costs at most twice the optimal tour, giving a 2-approximation immediately (Module 13).
- **Maze generation.** Assign random weights to a grid's edges and take the MST: the result is a perfect maze with exactly one path between any two cells.
- **Network design.** The original motivation — the cheapest set of links connecting all sites.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
MIT 6.046J — Greedy Algorithms, Minimum Spanning Trees	The cut property proved carefully, with both algorithms derived from it.
Jeff Erickson, Algorithms — Minimum Spanning Trees chapter	Presents Kruskal, Prim, and Borůvka as one generic algorithm. Worth reading for that unification alone.
MIT 6.006 — Union-Find / Disjoint Sets	The data structure with the amortised analysis at an accessible level.
CSES Problem Set — Graph section (Road Reparation, Road Construction)	Direct practice on MST and union-find.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement union-find four ways: neither optimisation, rank only, compression only, and both. Measure all four on 10^6 union and find operations and plot the results.

- 2 Implement Kruskal and Prim. Verify they produce the same total weight on 100 random graphs, and find a graph with equal weights where the two trees differ structurally.
- 3 Prove the cut property in your own words, then construct a graph with duplicate edge weights where the MST is genuinely not unique.
- 4 Implement single-linkage clustering by building an MST and removing the $k-1$ heaviest edges. Test it on 2D point data and plot the clusters.
- 5 Generate a maze by assigning random weights to a grid graph and taking the MST. Verify there is exactly one path between any two cells.
- 6 Measure Kruskal against Prim on a sparse graph ($E = 2V$) and a dense one ($E = V^2/2$). Confirm the predicted crossover.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State the cut property and prove it with an exchange argument.
- 2 Show how the cut property applies to Prim and to Kruskal, naming the cut in each case.
- 3 Why is Prim essentially Dijkstra? What exactly differs?
- 4 Why does union-find need both path compression and union by rank?
- 5 What is $\alpha(n)$ and in what sense is union-find constant time but not constant time?
- 6 What is a matroid, and what does the Rado–Edmonds theorem tell you about when greedy algorithms work?